Module 5 — Testing, CI/CD, GitHub Actions and Deployment

Vistula University · IT Fundamentals · Theory handout

Vistula University · IT Fundamentals

Learning Objectives

After this module, you will be able to:

- Explain why software needs testing, and why bugs cost more when we find them late.
- Tell the difference between unit tests, integration tests, and end-to-end tests.
- Compare manual testing and automated testing, and say when each one is useful.
- Write a simple test case with an input, steps, an expected result, and an actual result.
- Use basic assertions, and design positive and negative test scenarios.
- Explain what CI (Continuous Integration) and CD (Continuous Delivery) mean.
- Read a simple GitHub Actions workflow and describe its triggers, jobs, and steps.
- Explain production vs development environments, and authentication vs authorization.

---

1. Why Software Needs Testing

A **bug** is a mistake in a program. The program does something wrong, or it stops working. All programmers make bugs — even very good ones. This is normal. The question is not "will we have bugs?" but "when will we find them?"

Bugs cost more when we find them late. Think about our **Student Task Manager** project:

- We find a bug in `formatTaskTitle()` while we are writing it → we fix it in one minute.
- We find the same bug one week later, in the finished page → we must understand old code, fix it, and test again. Maybe one hour.
- A student user finds it → tasks show wrong titles on the live website, users lose trust. This costs the most.

So the rule is simple: **test early, test often**. Good news: computers can run most checks for us, again and again, for free.

2. Levels of Testing

Unit tests — test one small piece

A **unit** means one small piece of code, usually one function. A **unit test** checks that this one piece works alone. Example: `formatTaskTitle(" Buy milk ")` should return `"Buy milk"`. We test only this function — not the page, not the database. Unit tests are fast and easy to write.

Integration tests — test pieces working together

An **integration test** checks that two or more pieces work together. Example: when the user clicks "Add task", the function runs and the task is saved to Supabase. Here we test the connection between parts, not each part alone.

End-to-end tests — test the whole user flow

An **end-to-end (E2E) test** checks a full user journey, like a real user. Example: open the page in a browser → log in → add a task → mark it done → log out. The name means "from one end of the program to the other end". These tests are the slowest, but they check what the user really sees.

Rule of thumb: many unit tests, fewer integration tests, a few end-to-end tests.

3. Manual vs Automated Testing

In **manual testing**, a person uses the program with their hands and eyes and writes down what happens. In **automated testing**, a script runs the checks by itself.

	Manual testing	Automated testing
Who does it	A person	A computer script
Speed	Slow	Very fast (seconds)
Repeating the same test 100 times	Boring, mistakes happen	Easy, no mistakes
Best for	How the page looks and feels; one-time checks	Checking the same rules after every change
Cost over time	Grows (more people needed)	Small after writing the test once

Good teams use both: automated tests for the rules, manual clicks for the "feel" of the page.

4. The Test Case: Input, Steps, Expected vs Actual Result

A **test case** is a written plan for one check. Every test case has:

- **Input** — the data we give to the program.
- **Steps** — what we do.
- **Expected result** — what *should* happen.
- **Actual result** — what *really* happened when we tried.

If expected = actual → the test **passes**. If they are different → the test **fails**, and we found a bug.

Example for our Task Manager:

#	Input	Expected result	Actual result	Pass?
1	`" Buy milk "`	`"Buy milk"` (spaces removed)	`"Buy milk"`	✅
2	`""` (empty)	error, task not added	task was added	❌ bug!

5. Assertions — the Check Inside a Test

An **assertion** is a line of code that says "I expect X, and if it is not true, the test fails". It is the heart of an automated test. A common JavaScript style:

```
const result = formatTaskTitle("  Buy milk  ");
expect(result).toBe("Buy milk"); // the assertion
```

If the result is wrong, the test fails loudly. No assertion = no real test, because nothing checks the answer.

6. Positive and Negative Test Scenarios

- A **positive test** uses **valid** (correct) input and checks that the program succeeds. Example: title `"Read a book"` → the task is added.
- A **negative test** uses **invalid** (wrong) input and checks that the program handles it politely. Example: empty title `""` → the program shows an error and does **not** add the task.

Beginners test only the happy path. But users type strange things! Always ask: "What happens if the input is empty, too long, or weird?" A good program stays calm with bad input.

7. CI — Continuous Integration

CI means **Continuous Integration**. "Continuous" = all the time, with every change. "Integration" = putting new code together with the rest of the project.

With CI, every time you **push** (send your code to GitHub), the server automatically:

1. takes a copy of your code,
2. **builds** it (prepares it to run),
3. runs all the tests.

If any test fails, the team sees a red mark immediately and fixes the problem while the code is still fresh. Without CI, everyone works alone for weeks, and joining the code later breaks everything.

8. CD — Continuous Delivery / Deployment

CD means **Continuous Delivery** or **Continuous Deployment**. It is the next step after CI: every change that passes the tests can be **released** — given to users — automatically.

- **Continuous Delivery**: the new version is packaged and ready; a person presses one button to publish it.
- **Continuous Deployment**: even that button disappears — if all tests pass, the new version goes live by itself.

For our Task Manager: you push code → GitHub tests it → if green, the site on GitHub Pages updates automatically.

9. GitHub Actions — Automation in Your Repository

GitHub Actions is a tool inside GitHub that runs your CI/CD. You write a **workflow** — a file that describes what to do and when.

A workflow has three main parts:

- **Trigger** — the event that starts it, for example `on: push` ("every time someone pushes").
- **Jobs** — big blocks of work, for example "test the code".
- **Steps** — small actions inside a job: check out the code, install Node, run tests.

Here is a **minimal working workflow** for a small Node project. Save it as `.github/workflows/ci.yml`:

```
name: CI
on: push

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: node js/utils.test.js
```

Reading it: the workflow runs on every push; one job called `test` runs on a fresh Linux machine; the steps download the repository (`checkout`), install Node 20, and run our test file. Afterwards, GitHub shows a green check ✓ or a red cross ✗ next to your commit.

10. Build and Deployment

- **Build** — preparing the code to run: checking it, joining files, making it small and fast. Our simple site needs almost no build.
- **Deployment** — putting the built application on a server so users can open it in a browser. For us, the "server" is GitHub Pages.

Build comes first, deployment comes second. You deploy a build, not your working folder.

11. Production vs Development

A **development environment** is where you work: your own computer, with test data and tools for finding errors. The **production environment** is the live application that real users open. Mistakes there are public!

	Development	**Production**
Who uses it	You, the programmer	Real users
Where	Your computer (localhost)	Server (GitHub Pages, internet)
Data	Fake, test data	Real data
Errors	Normal, expected	Cost trust and money
Changes	Any time, any moment	Carefully, after tests

Golden rule: **never test on production** — always test in development first.

12. Basic Security Concepts

Authentication vs Authorization

- **Authentication** answers: **who are you?** It is the login. Example: you enter your email and password in Supabase, and the system says "this is Maria, student".
- **Authorization** answers: **what may you do?** It checks permissions *after* login. Example: Maria may edit **her own** tasks, but not the tasks of other students.

You can be authenticated (logged in) but not authorized (not allowed) to do something. Both checks are needed.

Secrets

A **secret** is private data like a password, an API key, or a database token. **Never put secrets in your code or push them to GitHub** — anyone can read a public repository. Keep them in special settings (environment variables) outside the code.

HTTPS

HTTPS means the connection between the browser and the server is **encrypted** (scrambled). Nobody in the middle — for example on public Wi-Fi — can read your password. Always use `https://` for anything with logins.

13. Why Code Review and Tests Help a Small Team

Code review means: another person reads your code before it is merged and suggests better ways. In a small team this pays off:

- Two pairs of eyes find more bugs than one.
- Everyone learns how the whole project works.
- Automated tests protect old features: when you change something, the tests tell you immediately if you broke something else.

Together: review + tests + CI = a small team can move fast **and** stay safe.

Key Terms (Glossary)

Term	Simple meaning (English)	Po polsku
unit test	A test of one small piece of code, usually one function.	test jednostkowy
integration test	A test that checks if several pieces work together.	test integracyjny
end-to-end test	A test of a whole user journey in the program, like a real user.	test end-to-end (od początku do końca)
manual testing	A person checks the program by hand.	testowanie ręczne
automated testing	A script checks the program automatically.	testowanie automatyczne
test case	A written check: input, steps, expected result, actual result.	przypadek testowy
assertion	A line of code that checks “is the result what I expect?”	asercja
positive test	A test with valid input; the program should succeed.	test pozytywny
negative test	A test with invalid input; the program should handle the error.	test negatywny
CI	Continuous Integration — every push is built and tested automatically.	ciągła integracja
CD	Continuous Delivery/Deployment — changes are released automatically.	ciągłe dostarczanie / wdrażanie
workflow (GitHub Actions)	A file that says what to run and when (trigger, jobs, steps).	przepływ pracy

Term	Simple meaning (English)	Po polsku
build	Preparing the code to run (checking, joining, optimizing files).	budowanie (kompilacja)
deployment	Putting the application on a server so users can open it.	wdrożenie
production	The live application that real users use.	środowisko produkcyjne
authentication	Checking who you are (login).	uwierzytelnianie
authorization	Checking what you may do (permissions).	autoryzacja
secret	Private data (password, key); never put it in code.	sekret / tajny klucz
HTTPS	Encrypted connection between browser and server.	HTTPS (szyfrowane połączenie)
code review	Another person reads your code before merging.	przegląd kodu

Summary

- Bugs are normal; finding them early is much cheaper than finding them late.
- Unit tests check one function; integration tests check pieces together; end-to-end tests check the whole user flow.
- Manual testing is a person clicking; automated testing is a script running the same checks fast.
- A test case = input + steps + expected result + actual result; the assertion is the check inside.
- Test both sides: positive tests (valid input) and negative tests (invalid input, like an empty task title).
- CI means every push is automatically built and tested; CD means changes can go to users automatically.
- A GitHub Actions workflow has a trigger (`on: push`), jobs, and steps; deployment puts the built app on a server.
- Production is the live site — never test there. Authentication = who you are; authorization = what you may do.